

UNIVERSIDAD PRIVADA “FRANZ TAMAYO”

FACULTAD DE INGENIERIA

INGENIERÍA EN SISTEMAS



“SignBridge AI”

ESTUDIANTE:

ADRIAN DARIO MAMANI CUTILE

DOCENTE:

ING.JAVIER ELVIS CANQUI LLUSCO

MATERIA:

INGENIERIA DE SISTEMAS

LA PAZ - EL ALTO – BOLIVIA

2026

INDICE

TABLA DE CONTENIDOS	¡Error! Marcador no definido.
CAPÍTULO 1: MARCO INTRODUCTORIO	5
1.1 Introducción	5
1.2 Antecedentes	5
1.3 Planteamiento del Problema	5
1.4 Formulación del Problema.....	6
1.5 Objetivo General	6
1.6 Objetivos Específicos.....	6
1.7 Alcance del Proyecto	7
1.8 Justificación	7
1.8.1 Justificación Tecnológica.....	7
1.8.2 Justificación Social	8
1.8.3 Justificación Académica	8
1.8.4 Justificación Económica	8
1.9 Metodología de Desarrollo	8
1.9.1 Scrum en SignBridge AI.....	9
1.9.2 Sprint Planning.....	9
1.9.3 Sprint Review.....	9
1.9.4 Sprint Retrospective.....	9
1.9.5 Product Backlog.....	9
1.9.6 Sprint Backlog	9
CAPÍTULO 2: MARCO TEÓRICO.....	10
2.1 Inteligencia Artificial	10
2.2 Machine Learning	10
2.3 Deep Learning.....	10
2.4 Visión por Computadora.....	10
2.5 TensorFlow	10
2.6 Redes Neuronales Convolucionales (CNN).....	11
2.7 Flask / FastAPI.....	11
2.8 Arquitectura Cliente-Servidor.....	11
2.9 APIs REST.....	11
2.10 HTML5	11

2.11 CSS3	12
2.12 JavaScript.....	12
2.13 Git y GitHub	12
2.14 GitHub Pages	12
2.15 Accesibilidad Digital	12
2.16 Inclusión Tecnológica.....	13
2.17 Lengua de Señas Boliviana (LSBO).....	13
2.18 Sistemas Inteligentes de Traducción.....	13
CAPÍTULO 3: MARCO DE DESARROLLO	14
3.1 Descripción General del Sistema.....	14
3.2 Arquitectura del Sistema.....	14
3.3 Requerimientos Funcionales	14
3.4 Requerimientos No Funcionales	15
3.5 Historias de Usuario.....	16
3.6 Casos de Uso.....	17
Escenario de Éxito Detallado — CU02: Detectar y Clasificar Señal	18
3.7 Diagramas UML	18
3.7.1 Diagrama de Casos de Uso	19
3.7.2 Diagrama de Clases.....	19
3.7.3 Diagrama de Secuencia.....	20
3.7.4 Diagrama de Componentes	21
3.7.5 Diagrama de Despliegue	22
3.8 Modelo Entidad-Relación (DER)	22
3.9 Diccionario de Datos.....	24
3.10 Matriz de Trazabilidad (RTM).....	25
3.10.1 Cadena de Trazabilidad del Formulario de Contacto (Landing Page).....	26
3.11 Landing Page	26
3.11.1 Estructura de la Landing Page	26
3.11.2 Diseño Responsivo y Accesibilidad.....	27
3.12 Implementación Técnica.....	27
3.12.1 Pipeline de Procesamiento en Tiempo Real.....	27
3.12.2 Modelo CNN de Clasificación.....	27
3.12.3 Comunicación WebSocket.....	27
3.12.4 API REST	27

3.13 Despliegue del Proyecto	27
3.13.1 Repositorio GitHub.....	27
3.13.2 Configuración de GitHub Pages	28
3.13.3 URLs del Proyecto.....	28
3.14 Resultados Obtenidos.....	29
3.15 Conclusiones	29
3.16 Recomendaciones	30
SECCIÓN DE CIERRE: ARTEFACTOS REUTILIZADOS Y ACTUALIZADOS	32
BIBLIOGRAFÍA	34

CAPÍTULO 1: MARCO INTRODUCTORIO

1.1 Introducción

En la actualidad, la inclusión tecnológica representa uno de los desafíos más relevantes para el desarrollo social equitativo. Las personas con discapacidad auditiva enfrentan barreras comunicativas significativas en entornos educativos, laborales y sociales, principalmente como consecuencia de la escasa disponibilidad de herramientas tecnológicas accesibles que medien entre la lengua de señas y el lenguaje oral o escrito.

SignBridge AI surge como respuesta a esta problemática, constituyendo un sistema inteligente de reconocimiento automático y traducción en tiempo real de la Lengua de Señas Boliviana (LSBO). El sistema integra técnicas avanzadas de visión por computadora, aprendizaje profundo mediante redes neuronales convolucionales (CNN) y síntesis de voz, con el propósito de democratizar el acceso a la comunicación para la comunidad sorda y con discapacidad auditiva en Bolivia.

El presente informe corresponde al Hito 4 del Proyecto Integrador Intermedio I, y consolida el ciclo completo de ingeniería de software aplicado al sistema SignBridge AI: desde la especificación de requerimientos y el modelado arquitectónico, hasta la implementación técnica, el despliegue en producción mediante GitHub Pages y la trazabilidad integral entre los artefactos del proyecto.

1.2 Antecedentes

El reconocimiento automático de lengua de señas (Sign Language Recognition, SLR) constituye un campo de investigación activa dentro de la visión por computadora y la inteligencia artificial. Los primeros sistemas de reconocimiento gestual basados en guantes instrumentados datan de la década de 1980, mientras que los enfoques basados en visión artificial comenzaron a consolidarse con el advenimiento del aprendizaje profundo a partir de 2012, con la irrupción de AlexNet en el desafío ImageNet.

A nivel global, proyectos como el sistema de reconocimiento de lengua de señas americana (ASL) de Google, el trabajo de la Universidad de Oxford sobre BSL, y diversas iniciativas académicas en Latinoamérica han demostrado la viabilidad técnica de los sistemas de traducción gestual basados en CNN y frameworks de detección de landmarks como MediaPipe. En Bolivia, el desarrollo de herramientas tecnológicas orientadas a la LSBO es prácticamente inexistente en el ámbito del software de consumo masivo, lo que justifica plenamente la iniciativa del presente proyecto.

El equipo de desarrollo de SignBridge AI identificó, en etapas previas del proyecto integrador, la ausencia de soluciones locales accesibles para la comunidad sorda boliviana, y estableció como referentes tecnológicos los frameworks TensorFlow, MediaPipe y React, cuya combinación permite construir un sistema eficiente, de bajo costo computacional y desplegable en entornos web estándar sin requerir hardware especializado.

1.3 Planteamiento del Problema

En Bolivia, la comunidad de personas con discapacidad auditiva enfrenta una realidad marcada por la exclusión comunicativa sistémica. Según el Censo Nacional de 2012, la discapacidad auditiva afecta al 0.9% de la población boliviana, cifra que, proyectada al año 2026, representa aproximadamente 120,000 personas. A nivel global, la Organización Mundial de la Salud (OMS, 2023) estima que más de 430 millones de personas padecen pérdida auditiva discapacitante, proyectando que esta cifra superará los 700 millones hacia 2050.

La principal barrera de inclusión identificada radica en la escasa disponibilidad de intérpretes certificados de LSBO en Bolivia, cuyo costo oscila entre USD 60 y USD 120 por hora de servicio (OPS, 2021), haciendo inaccesible la mediación comunicativa profesional para la mayoría de la población afectada. Asimismo, las soluciones tecnológicas existentes en el mercado están diseñadas fundamentalmente para el reconocimiento de ASL (Lengua de Señas Americana) o LSE (Lengua de Señas Española), sin contemplar las particularidades léxicas y fonológicas de la LSBO.

Esta brecha tecnológica genera consecuencias directas en el acceso a la educación, la salud y el empleo por parte de las personas sordas bolivianas, perpetuando ciclos de exclusión social que contravienen los principios de la Convención sobre los Derechos de las Personas con Discapacidad (CDPD) de las Naciones Unidas, ratificada por Bolivia mediante Ley N.º 4024 de 2009.

1.4 Formulación del Problema

¿De qué manera el diseño e implementación de un sistema inteligente de reconocimiento y traducción en tiempo real de la Lengua de Señas Boliviana, basado en visión por computadora, redes neuronales convolucionales y síntesis de voz, puede contribuir a reducir las barreras comunicativas que enfrentan las personas con discapacidad auditiva en Bolivia, facilitando su inclusión en entornos educativos, laborales y sociales?

1.5 Objetivo General

Desarrollar un sistema de software inteligente denominado SignBridge AI que, mediante técnicas de visión por computadora, aprendizaje profundo y síntesis de voz, sea capaz de detectar, clasificar y traducir en tiempo real los gestos de la Lengua de Señas Boliviana capturados a través de una cámara web, generando como salida texto legible y audio sintetizado, con el propósito de reducir las barreras comunicativas de la comunidad sorda y con discapacidad auditiva en entornos educativos, laborales y sociales de Bolivia.

1.6 Objetivos Específicos

1. OE1 — Módulo de Captura: Implementar un módulo de captura de video en tiempo real que procese fotogramas a una tasa mínima de 24 FPS mediante la API de cámara web del navegador (WebRTC), garantizando una latencia de adquisición inferior a 50 ms bajo condiciones estándar de iluminación.
 2. OE2 — Módulo de Procesamiento con MediaPipe: Integrar la biblioteca MediaPipe Hands de Google para la detección y extracción de 21 puntos de referencia (landmarks) de la mano en coordenadas normalizadas (x, y, z), logrando una tasa de detección superior al 90% en condiciones controladas de prueba.
-

3. OE3 — Módulo de IA con TensorFlow: Diseñar, entrenar y optimizar un modelo de clasificación basado en redes neuronales convolucionales (CNN) utilizando TensorFlow 2.x, alcanzando una precisión de clasificación superior al 92% sobre el conjunto de validación para el alfabeto dactilológico y señas básicas de la LSBO.
4. OE4 — Módulo de Traducción Texto/Voz: Desarrollar un módulo de traducción que convierta la secuencia de señas clasificadas en texto estructurado y produzca síntesis de voz mediante la Web Speech API, con un tiempo de respuesta extremo a extremo (captura → traducción) inferior a 200 ms.
5. OE5 — Módulo de Interfaz Web: Construir una interfaz de usuario accesible, responsiva e intuitiva en React.js, conforme a las pautas WCAG 2.1 nivel AA, que permita visualizar la transmisión de video, la seña detectada, el texto traducido acumulado y los controles de sesión, con soporte para dispositivos de escritorio y móviles.

1.7 Alcance del Proyecto

El alcance del sistema SignBridge AI se delimita en los siguientes términos:

- Reconocimiento del alfabeto dactilológico completo de la LSBO (26 letras) y un vocabulario básico de señas compuesto por 20 palabras de alta frecuencia de uso (saludos, despedidas, números del 1 al 10, y señas de necesidad básica).
- Procesamiento de video en tiempo real mediante cámara web estándar, sin requerir hardware especializado adicional.
- Interfaz web accesible desplegada en navegadores modernos (Chrome 110+, Firefox 108+, Edge 110+, Safari 16+).
- Síntesis de voz en idioma español (variante boliviana) mediante Web Speech API.
- Landing Page institucional del proyecto, desplegada en GitHub Pages, con formulario de contacto funcional.
- Persistencia de sesiones, señas y traducciones en base de datos SQLite (entorno de desarrollo) y PostgreSQL (entorno de producción).

Quedan fuera del alcance de la versión 1.0:

- Reconocimiento de lenguaje de señas fluido o continuo (más allá de señas individuales).
- Traducción inversa (texto o voz a señas animadas).
- Autenticación de usuarios con roles diferenciados.
- Aplicación móvil nativa (iOS/Android).
- Soporte para variantes de lengua de señas distintas a la LSBO y ASL.

1.8 Justificación

1.8.1 Justificación Tecnológica

La confluencia de tecnologías maduras y de código abierto — TensorFlow 2.x, MediaPipe 0.10, React 18, FastAPI y WebRTC — permite construir un sistema de reconocimiento gestual de alta performance sin incurrir en costos de licenciamiento. La arquitectura cliente-servidor con comunicación via WebSocket garantiza baja latencia y escalabilidad horizontal. Adicionalmente, el uso de TensorFlow Lite para la serialización del modelo CNN posibilita la inferencia eficiente tanto en el servidor como en dispositivos de capacidad limitada, abriendo la puerta a versiones futuras con inferencia en el borde (edge computing).

1.8.2 Justificación Social

La Organización Mundial de la Salud (OMS, 2023) estima que más de 430 millones de personas en el mundo padecen pérdida auditiva discapacitante. En Bolivia, esta población enfrenta barreras estructurales de acceso a servicios básicos por la escasez de intérpretes certificados. SignBridge AI responde directamente a los artículos 9 y 21 de la Convención sobre los Derechos de las Personas con Discapacidad de las Naciones Unidas, que reconocen el derecho de acceso a la información y la comunicación. Al democratizar el acceso a herramientas de traducción de lengua de señas, el sistema fomenta la inclusión social, la autonomía comunicativa y la igualdad de oportunidades, alineándose con el ODS N.º 10 (Reducción de las Desigualdades) de la Agenda 2030 de las Naciones Unidas.

1.8.3 Justificación Académica

El desarrollo de SignBridge AI constituye un ejercicio integral de ingeniería de software que articula competencias en procesamiento de imágenes en tiempo real, diseño de arquitecturas de redes neuronales, ingeniería de requisitos, metodologías ágiles y diseño de interfaces accesibles. Como señala Pressman (2014), los proyectos de software complejos son el vehículo más efectivo para consolidar competencias en gestión de configuraciones y aseguramiento de calidad. El proyecto contribuye además al cuerpo de conocimiento en reconocimiento de gestos (gesture recognition), representando una arquitectura reproducible y escalable que puede servir de referencia para investigaciones posteriores en LSBO.

1.8.4 Justificación Económica

Los servicios de interpretación profesional de lengua de señas en América Latina tienen un costo promedio de entre USD 60 y USD 120 por hora (OPS, 2021), haciéndolos financieramente inaccesibles para la mayoría de la población con discapacidad auditiva. SignBridge AI opera sobre infraestructura de hardware convencional (cámara web estándar y navegador moderno), eliminando la necesidad de dispositivos especializados. Su modelo de código abierto permite el despliegue gratuito por parte de instituciones educativas y de salud, y su arquitectura modular es compatible con modelos de monetización freemium para sostenibilidad financiera a largo plazo.

1.9 Metodología de Desarrollo

El proyecto SignBridge AI fue desarrollado bajo la metodología Scrum, framework ágil de gestión de proyectos de software definido por Schwaber y Sutherland en la Guía Scrum 2020. A continuación se describen los eventos y artefactos Scrum aplicados específicamente al proyecto:

1.9.1 Scrum en SignBridge AI

Scrum es un framework empírico basado en tres pilares fundamentales: transparencia, inspección y adaptación. Su aplicación en SignBridge AI permitió estructurar el desarrollo en ciclos iterativos de duración fija (sprints), con entregas incrementales y verificables al final de cada ciclo, facilitando la retroalimentación temprana del docente-cliente y la adaptación continua del sistema a los requisitos emergentes.

1.9.2 Sprint Planning

Al inicio de cada sprint, el equipo de desarrollo realizó una sesión de Sprint Planning en la que se seleccionaron los ítems del Product Backlog con mayor prioridad y viabilidad técnica para el sprint en curso. Se estimaron los esfuerzos mediante Story Points y se definió el Sprint Goal: el objetivo concreto que el sprint debía alcanzar. En SignBridge AI, los Sprint Goals fueron: (Sprint 1) Configuración del entorno y prototipo de captura de video; (Sprint 2) Integración de MediaPipe y extracción de landmarks; (Sprint 3) Entrenamiento y validación del modelo CNN; (Sprint 4) Desarrollo de la interfaz React y módulo de síntesis de voz; (Sprint 5) Integración completa, Landing Page y despliegue en GitHub Pages.

1.9.3 Sprint Review

Al finalizar cada sprint, el equipo presentó el incremento funcional del software al docente-cliente. Se verificó el cumplimiento del Sprint Goal y se recopiló retroalimentación para ajustar el Product Backlog. En SignBridge AI, las Sprint Reviews permitieron identificar ajustes en la arquitectura de la capa de comunicación WebSocket (Sprint 2) y optimizaciones en el preprocesamiento de landmarks para mejorar la precisión del modelo (Sprint 3).

1.9.4 Sprint Retrospective

La Sprint Retrospective se realizó inmediatamente después de la Sprint Review en cada ciclo. El equipo analizó qué salió bien, qué se puede mejorar y qué acciones concretas se implementarán en el siguiente sprint. Las principales mejoras identificadas en SignBridge AI fueron: establecer convenciones de nombrado de variables (Sprint 1), adoptar el patrón Repository para la capa de datos (Sprint 3) y sistematizar las pruebas unitarias antes de la integración (Sprint 4).

1.9.5 Product Backlog

El Product Backlog de SignBridge AI contiene la lista priorizada y ordenada de todos los requisitos funcionales, historias de usuario y mejoras del sistema. Está mantenido por el Product Owner (representado por el docente en el contexto académico) y se actualiza continuamente mediante el proceso de refinamiento. Los ítems se expresan en formato de Historia de Usuario y se priorizan según valor de negocio, riesgo técnico y dependencias.

1.9.6 Sprint Backlog

El Sprint Backlog es el subconjunto de ítems del Product Backlog seleccionados para un sprint específico, descompuestos en tareas técnicas con estimación en horas. En SignBridge AI, el Sprint Backlog se gestionó mediante un tablero Kanban digital con columnas: Pendiente, En Progreso, En Revisión y Completado, permitiendo visibilidad en tiempo real del progreso del equipo.

CAPÍTULO 2: MARCO TEÓRICO

2.1 Inteligencia Artificial

La Inteligencia Artificial (IA) es la rama de la informática que estudia el diseño de sistemas capaces de realizar tareas que, ejecutadas por seres humanos, requieren inteligencia. Según Russell y Norvig (2020), la IA abarca el aprendizaje automático, el razonamiento lógico, la planificación, la percepción y el procesamiento del lenguaje natural. En el contexto de SignBridge AI, la IA se manifiesta en la capacidad del sistema para reconocer patrones gestuales complejos y traducirlos a lenguaje textual y auditivo en tiempo real, emulando la función cognitiva de un intérprete humano de lengua de señas.

2.2 Machine Learning

El Machine Learning (ML) o aprendizaje automático es una subdisciplina de la IA que dota a los sistemas de la capacidad de aprender y mejorar a partir de la experiencia, sin necesidad de programación explícita para cada tarea. Mitchell (1997) lo define como: 'un programa aprende de la experiencia E respecto a una clase de tareas T y medida de rendimiento P, si su rendimiento en T, medido por P, mejora con la experiencia E'. En SignBridge AI, el modelo CNN aprende a clasificar señas a partir de un dataset etiquetado de vectores de landmarks, mejorando su precisión con cada ciclo de entrenamiento.

2.3 Deep Learning

El Deep Learning (aprendizaje profundo) es una subdisciplina del ML que utiliza redes neuronales artificiales con múltiples capas (capas profundas) para aprender representaciones jerárquicas de los datos. LeCun, Bengio y Hinton (2015) describen el aprendizaje profundo como el método que ha revolucionado el reconocimiento de imágenes, voz y texto. En SignBridge AI, la arquitectura CNN profunda permite extraer automáticamente características discriminativas de los vectores de landmarks de la mano, sin necesidad de ingeniería manual de características.

2.4 Visión por Computadora

La visión por computadora es el campo de la IA que permite a los sistemas interpretar y comprender el contenido visual de imágenes y videos. Szeliski (2022) la define como el estudio de cómo las computadoras pueden adquirir, procesar y analizar imágenes digitales del mundo físico. En SignBridge AI, la visión por computadora se implementa en el pipeline de procesamiento de fotogramas de video, donde MediaPipe detecta la región de la mano y extrae sus puntos de referencia tridimensionales en tiempo real.

2.5 TensorFlow

TensorFlow es una biblioteca de código abierto desarrollada por Google Brain, diseñada para el cómputo numérico y el aprendizaje automático. En su versión 2.x, adopta la ejecución dinámica (eager execution) y la API de alto nivel Keras, facilitando el diseño, entrenamiento y despliegue de modelos de deep learning. En SignBridge AI, TensorFlow 2.x se utiliza para entrenar el modelo CNN de clasificación de señas, mientras que TensorFlow

Lite proporciona un formato optimizado para inferencia eficiente en tiempo real dentro del servidor FastAPI.

2.6 Redes Neuronales Convolucionales (CNN)

Las Redes Neuronales Convolucionales (Convolutional Neural Networks, CNN) son una clase de redes neuronales profundas especialmente diseñadas para el procesamiento de datos con estructura de cuadrícula, como imágenes. Según LeCun et al. (1998), las CNN utilizan la operación de convolución para extraer características locales mediante filtros aprendibles, seguida de capas de pooling para reducción de dimensionalidad. En SignBridge AI, la CNN recibe el vector normalizado de 63 valores float32 (21 landmarks $\times$ 3 coordenadas) y produce un vector de probabilidades softmax sobre las N clases de señas del vocabulario.

2.7 Flask / FastAPI

El backend de SignBridge AI está construido sobre FastAPI, un framework moderno de Python para la construcción de APIs web de alto rendimiento, basado en los estándares OpenAPI y JSON Schema. FastAPI utiliza anotaciones de tipo de Python (type hints) para generar automáticamente documentación interactiva (Swagger UI y ReDoc) y proporciona soporte nativo para operaciones asíncronas mediante asyncio. A diferencia del más conocido Flask, FastAPI ofrece validación automática de datos de entrada, serialización JSON eficiente mediante Pydantic y soporte para WebSockets, características críticas para el pipeline de procesamiento en tiempo real de SignBridge AI.

2.8 Arquitectura Cliente-Servidor

La arquitectura cliente-servidor es un modelo de computación distribuida en el que el cliente solicita servicios y el servidor los provee. En SignBridge AI, el cliente es la aplicación React corriendo en el navegador del usuario, y el servidor es la instancia FastAPI que expone endpoints REST y WebSocket. Esta separación garantiza la independencia tecnológica entre las capas de presentación y negocio, facilita el escalado horizontal del servidor y permite que múltiples clientes accedan concurrentemente a los servicios de reconocimiento de señas.

2.9 APIs REST

REST (Representational State Transfer) es un estilo arquitectónico para sistemas distribuidos propuesto por Fielding (2000), que define un conjunto de restricciones para el diseño de APIs web. Las APIs REST de SignBridge AI exponen endpoints para la gestión de sesiones (POST /sesiones, GET /sesiones/{id}), la exportación de historiales (GET /sesiones/{id}/exportar) y el registro de contactos desde la Landing Page (POST /contactos). La comunicación en tiempo real para el streaming de frames se implementa mediante WebSocket (ws://), complementando la API REST.

2.10 HTML5

HTML5 es la quinta revisión del estándar HTML (HyperText Markup Language), que introduce elementos semánticos (<header>, <main>, <section>, <article>, <nav>), la API de canvas 2D para renderizado gráfico, la API de video/audio nativa, WebRTC para comunicación en tiempo real y el API de almacenamiento local. En SignBridge AI, HTML5

proporciona el soporte estructural de la Landing Page desplegada en GitHub Pages, incluyendo el canvas sobre el que se superponen los landmarks detectados en tiempo real.

2.11 CSS3

CSS3 (Cascading Style Sheets, nivel 3) introduce capacidades avanzadas de presentación visual, incluyendo transiciones, animaciones, variables CSS (custom properties), Flexbox, CSS Grid y media queries para diseño responsivo. En SignBridge AI, Tailwind CSS — un framework utility-first basado en CSS3 — se utiliza en el frontend React para implementar el sistema de diseño con paleta de colores en tono oscuro (dark navy #0D1B2A, cyan #00D4FF, verde #00FF9F), garantizando consistencia visual y adaptabilidad a distintos tamaños de pantalla.

2.12 JavaScript

JavaScript es el lenguaje de programación interpretado, orientado a eventos y con capacidades asíncronas que constituye el estándar de la web del lado del cliente. En SignBridge AI, JavaScript (en su variante TypeScript mediante React 18) gestiona el ciclo de vida de los componentes de la interfaz, la comunicación WebSocket con el backend, la captura de frames mediante la API getUserMedia de WebRTC, el dibujado de landmarks sobre el canvas HTML5 y la integración con la Web Speech API para síntesis de voz.

2.13 Git y GitHub

Git es un sistema de control de versiones distribuido (DVCS) creado por Linus Torvalds en 2005, que permite rastrear cambios en el código fuente y coordinar el trabajo entre múltiples desarrolladores. GitHub es la plataforma de alojamiento de repositorios Git más utilizada en el mundo, que añade herramientas de colaboración (pull requests, issues, actions). El repositorio de SignBridge AI está alojado en GitHub y utiliza un flujo de trabajo GitFlow con ramas main, develop y feature/*. El CI/CD se implementa mediante GitHub Actions para automatizar las pruebas unitarias y el despliegue de la Landing Page en GitHub Pages.

2.14 GitHub Pages

GitHub Pages es un servicio de alojamiento web estático proporcionado por GitHub de forma gratuita, que publica automáticamente el contenido del directorio raíz o de la carpeta /docs de un repositorio como un sitio web accesible públicamente. En SignBridge AI, GitHub Pages aloja la Landing Page del proyecto (index.html y assets asociados), proporcionando una URL pública estable para la presentación y difusión del sistema. El proceso de publicación se automatiza mediante GitHub Actions, garantizando que cada cambio en la rama main se refleje automáticamente en el sitio desplegado.

2.15 Accesibilidad Digital

La accesibilidad digital hace referencia al diseño de productos y servicios digitales que pueden ser utilizados por personas con diversas capacidades y discapacidades. Las Pautas de Accesibilidad para el Contenido Web (WCAG) 2.1, publicadas por el W3C, establecen tres niveles de conformidad (A, AA, AAA) y cuatro principios fundamentales: perceptible, operable, comprensible y robusto. SignBridge AI implementa el nivel AA de WCAG 2.1,

garantizando contraste de color suficiente, navegación por teclado, compatibilidad con lectores de pantalla y textos alternativos en todos los elementos visuales.

2.16 Inclusión Tecnológica

La inclusión tecnológica es el principio según el cual el diseño y desarrollo de tecnología debe contemplar las necesidades de todos los segmentos de la población, incluyendo aquellos históricamente excluidos por razones de discapacidad, origen socioeconómico o ubicación geográfica. SignBridge AI encarna este principio al ofrecer una herramienta de comunicación asistiva de código abierto, gratuita y desplegable en hardware de bajo costo, alineándose con los Objetivos de Desarrollo Sostenible de la Agenda 2030 de las Naciones Unidas, específicamente el ODS 10 (Reducción de Desigualdades) y el ODS 4 (Educación de Calidad).

2.17 Lengua de Señas Boliviana (LSBO)

La Lengua de Señas Boliviana (LSBO) es la lengua natural de la comunidad sorda de Bolivia, reconocida oficialmente por la Constitución Política del Estado Plurinacional de Bolivia de 2009 (Art. 30, numeral I). Es una lengua visual-gestual con gramática propia e independiente del español, que utiliza la forma, posición, orientación y movimiento de las manos, junto con expresiones faciales y corporales, para transmitir significado. El alfabeto dactilológico de la LSBO comparte características con el español (alfabeto de 27 letras) y constituye el punto de entrada más accesible para sistemas de reconocimiento automático como SignBridge AI.

2.18 Sistemas Inteligentes de Traducción

Los sistemas inteligentes de traducción de lengua de señas combinan técnicas de visión por computadora, aprendizaje automático y procesamiento del lenguaje natural para automatizar la mediación comunicativa entre sordos y oyentes. Koller (2020) clasifica estos sistemas en tres categorías: reconocimiento aislado de señas (como SignBridge AI v1.0), reconocimiento de señas continuas y traducción de lengua de señas completa. El enfoque de SignBridge AI —basado en landmarks de MediaPipe como representación intermedia— reduce significativamente los requisitos computacionales respecto a los sistemas basados en análisis de imagen cruda, manteniendo un balance óptimo entre precisión y eficiencia para entornos de recursos limitados.

CAPÍTULO 3: MARCO DE DESARROLLO

3.1 Descripción General del Sistema

SignBridge AI es un sistema web de tiempo real suave (soft real-time) que traduce gestos de la Lengua de Señas Boliviana a texto y voz mediante inteligencia artificial. El usuario accede al sistema a través de un navegador web moderno, activa su cámara web y realiza señas frente al dispositivo. El sistema procesa el flujo de video en tiempo real, detecta la mano mediante MediaPipe, extrae 21 puntos de referencia tridimensionales, los clasifica con un modelo CNN entrenado con TensorFlow, y presenta el resultado como texto en pantalla y voz sintetizada. La Landing Page del proyecto, desplegada en GitHub Pages, sirve como punto de entrada público para el sistema.

Siguiendo el marco de clasificación de Sommerville (2016), SignBridge AI se categoriza como un sistema de software interactivo en tiempo real con componentes de inteligencia artificial embebida, de criticidad no crítica (nivel C) y escala mediana (entre 10K–100K líneas de código estimadas).

3.2 Arquitectura del Sistema

SignBridge AI implementa una arquitectura de tres capas (Three-Tier Architecture) conforme al patrón descrito por Pressman (2014), separando las responsabilidades de presentación, lógica de negocio y persistencia de datos:

- Capa de Presentación: React.js 18 + Tailwind CSS. Componentes principales: VideoCapture.jsx, TranslationPanel.jsx, ControlBar.jsx, SignConfidence.jsx, HistorialSesiones.jsx y Configuracion.jsx. Comunicación con el backend vía WebSocket para streaming bidireccional y REST para operaciones CRUD.
- Capa de Lógica de Negocio: Python 3.11 + FastAPI + MediaPipe 0.10 + TensorFlow Lite 2.x. Módulos: capture_service.py, landmark_extractor.py, classifier_service.py, translation_service.py, session_manager.py, export_service.py. Expone API REST y WebSocket.
- Capa de Datos: SQLite (desarrollo) / PostgreSQL (producción) + SQLAlchemy ORM. Repositorios: UserRepository, SessionRepository, SignRepository, TranslationRepository. Los modelos .tflite se almacenan en sistema de archivos con registro en la entidad ModeloIA.

3.3 Requerimientos Funcionales

Los requisitos funcionales han sido redactados siguiendo el estándar IEEE 830 y priorizados mediante la técnica MoSCoW (Clegg y Barker, 1994), garantizando verificabilidad y medibilidad en cada caso:

ID	Descripción del Requisito	Criterio de Aceptación	MoSCoW	OE
RF-01	El sistema DEBE capturar y procesar el video de la cámara web a un mínimo de 24 FPS en resolución 640×480 px.	Medido con herramientas de profiling: ≥ 24 FPS en el 95% de los frames durante 60 s.	Must Have	OE1
RF-02	El sistema DEBE detectar la mano del usuario y extraer 21 landmarks normalizados mediante MediaPipe Hands.	Tasa de detección $\geq 90\%$ en dataset de 500 imágenes bajo iluminación estándar.	Must Have	OE2
RF-03	El sistema DEBE clasificar la seña detectada mediante el modelo CNN y mostrar el resultado con confianza $\geq 85\%$.	Precisión $\geq 92\%$ en conjunto de validación; umbral de confianza configurable.	Must Have	OE3
RF-04	El sistema DEBE traducir la secuencia de señas a texto y sintetizar audio en español con latencia <200 ms.	Prueba extremo a extremo: 95% de los casos bajo 200 ms en entorno estándar.	Must Have	OE4
RF-05	El sistema DEBERÍA permitir exportar el historial de traducción de la sesión en formato TXT o PDF.	Función disponible en la interfaz; archivo generado en <3 s para hasta 500 señas.	Should Have	OE5

Tabla 3.1. Requisitos Funcionales del sistema SignBridge AI con priorización MoSCoW.

3.4 Requerimientos No Funcionales

Los requisitos no funcionales definen los atributos de calidad del sistema según el estándar ISO/IEC 25010:

ID	Atributo de Calidad	Descripción y Criterio de Medición	MoSCoW
RNF-01	Rendimiento / Latencia	El pipeline completo (captura $\rightarrow$ clasificación $\rightarrow$ visualización) no debe superar 200 ms en el percentil 95 sobre hardware con CPU Intel i5 8. ^a gen. o superior y 8 GB RAM.	Must Have
RNF-02	Accesibilidad	La interfaz web debe cumplir WCAG 2.1 nivel AA, verificado con axe-core y evaluación manual.	Should Have
RNF-03	Disponibilidad	El sistema debe estar disponible el 99% del tiempo durante las horas de operación (8:00–22:00).	Must Have
RNF-04	Seguridad	Las comunicaciones entre frontend y backend deben realizarse sobre HTTPS/WSS; ningún dato biométrico se transmite sin cifrado.	Must Have
RNF-05	Usabilidad	Un usuario nuevo debe poder iniciar una sesión de traducción en menos de 3 minutos sin instrucciones adicionales.	Should Have
RNF-06	Escalabilidad	La arquitectura debe soportar al menos 50 sesiones simultáneas sin degradación perceptible del rendimiento.	Could Have
RNF-07	Mantenibilidad	El código debe mantener una cobertura de pruebas unitarias $\geq 70\%$ y seguir los estándares PEP 8 (Python) y ESLint (JS).	Should Have

ID	Atributo de Calidad	Descripción y Criterio de Medición	MoSCoW
RNF-08	Portabilidad	El sistema debe ser compatible con Chrome 110+, Firefox 108+, Edge 110+ y Safari 16+ sin plugins adicionales.	Must Have
RNF-09	Privacidad	Los frames de video procesados no deben almacenarse en disco; solo los landmarks extraídos pueden persistirse.	Must Have
RNF-10	Internacionalización	La interfaz debe soportar español (es-BO) como idioma principal, con arquitectura preparada para i18n futuro.	Could Have

Tabla 3.2. Requisitos No Funcionales del sistema SignBridge AI.

3.5 Historias de Usuario

Las historias de usuario se han redactado siguiendo el formato Scrum estándar y alineadas con los objetivos específicos del proyecto:

Historia de Usuario HU-01

Como persona con discapacidad auditiva,
quiero activar la cámara web y comenzar una sesión de traducción de señas,
para puedo comunicarme con personas oyentes sin necesidad de un intérprete externo.

Criterios de Aceptación:

- El botón 'Iniciar Sesión' activa la cámara en menos de 3 segundos.
- El sistema muestra confirmación visual de que la cámara está activa.
- El pipeline de reconocimiento comienza automáticamente al activar la cámara.

Historia de Usuario HU-02

Como persona con discapacidad auditiva,
quiero que el sistema detecte y clasifique mis señas en tiempo real con indicador de confianza,
para puedo saber si el sistema está reconociendo correctamente mis gestos y ajustar mi ejecución.

Criterios de Aceptación:

- La seña clasificada se muestra en pantalla en menos de 200 ms desde la realización del gesto.
- El indicador de confianza muestra el porcentaje de certeza del modelo (0–100%).
- Solo se registran señas con confianza $\geq 85\%$ en el historial de la sesión.

Historia de Usuario HU-03

Como persona con discapacidad auditiva,
quiero escuchar en voz alta el texto traducido de mis señas,

para poder comunicarme en entornos donde el texto en pantalla no es suficiente.

Criterios de Aceptación:

- El texto traducido se sintetiza en audio en menos de 500 ms desde la clasificación de la señal.
- El usuario puede ajustar la velocidad y volumen de síntesis desde el panel de configuración.
- La síntesis funciona sin conexión a internet mediante la Web Speech API del navegador.

Historia de Usuario HU-04

Como usuario del sistema,

quiero exportar el historial de traducción de mi sesión en formato TXT o PDF,

para poder conservar un registro de la comunicación realizada.

Criterios de Aceptación:

- El botón 'Exportar' está disponible durante y después de la sesión activa.
- El archivo generado incluye fecha, hora y secuencia completa de señales y textos traducidos.
- El archivo se descarga en menos de 3 segundos para sesiones de hasta 500 señales.

Historia de Usuario HU-05

Como visitante de la Landing Page,

quiero enviar un mensaje de contacto a los desarrolladores de SignBridge AI,

para poder obtener más información o reportar problemas con el sistema.

Criterios de Aceptación:

- El formulario de contacto valida nombre, email válido y mensaje no vacío antes de enviar.
- El usuario recibe confirmación visual inmediata tras el envío exitoso.
- El registro de contacto queda almacenado en la base de datos del sistema.

3.6 Casos de Uso

Los casos de uso del sistema SignBridge AI identifican los actores y las interacciones funcionales principales:

ID	Nombre del Caso de Uso	Actor Principal	Descripción Resumida
CU01	Iniciar Sesión de Traducción	Usuario Final	El usuario activa la cámara y el sistema inicia el pipeline de reconocimiento en tiempo real.
CU02	Detectar y Clasificar Señal	Motor de IA (TF + MediaPipe)	El sistema detecta la mano, extrae 21 landmarks y clasifica la señal con confianza $\geq 85\%$.
CU03	Traducir Señal a Texto	Sistema / Usuario	Las señales clasificadas se convierten en texto acumulado visible en pantalla.
CU04	Sintetizar Audio	Web Speech API	El texto traducido se convierte en voz mediante la API de síntesis de voz del navegador.
CU05	Exportar Historial	Usuario Final	El usuario descarga el historial de la sesión en formato TXT o PDF.
CU06	Detener Sesión	Usuario Final	El usuario cierra la sesión de traducción, liberando el recurso de la cámara web.
CU07	Configurar Preferencias	Usuario Final	El usuario ajusta umbral de confianza, velocidad de síntesis y variante de señales (LSB/ASL).

Tabla 3.3. Casos de uso principales del sistema SignBridge AI.

Escenario de Éxito Detallado — CU02: Detectar y Clasificar Señal

Elemento	Detalle
Precondición	El usuario ha iniciado sesión (CU01); la cámara web está activa y transmitiendo video a 24+ FPS.
Disparador	El módulo de captura entrega un fotograma normalizado al pipeline de procesamiento.
Paso 1	El módulo de captura convierte el fotograma BGR a RGB y lo normaliza a 640×480 px.
Paso 2	MediaPipe Hands detecta la región de la mano con el modelo BlazePalm.
Paso 3	MediaPipe extrae los 21 landmarks tridimensionales (x, y, z) normalizados respecto al bounding box.
Paso 4	El vector de landmarks (63 valores float32) se normaliza respecto al punto de muñeca (landmark 0) para invarianza de posición.
Paso 5	El vector normalizado se introduce al modelo CNN serializado en TensorFlow Lite.
Paso 6	El modelo produce un vector softmax sobre N clases; se selecciona la clase con $p \geq 0.85$.
Paso 7	La etiqueta se envía al módulo de traducción (CU03) y se visualiza en la UI con indicador de confianza.
Postcondición	La señal queda registrada en el buffer de sesión; el pipeline procesa el siguiente fotograma.
Flujo Alternativo	Si MediaPipe no detecta mano ($p < \text{umbral}$), el sistema muestra 'Sin señal detectada' y descarta el frame.

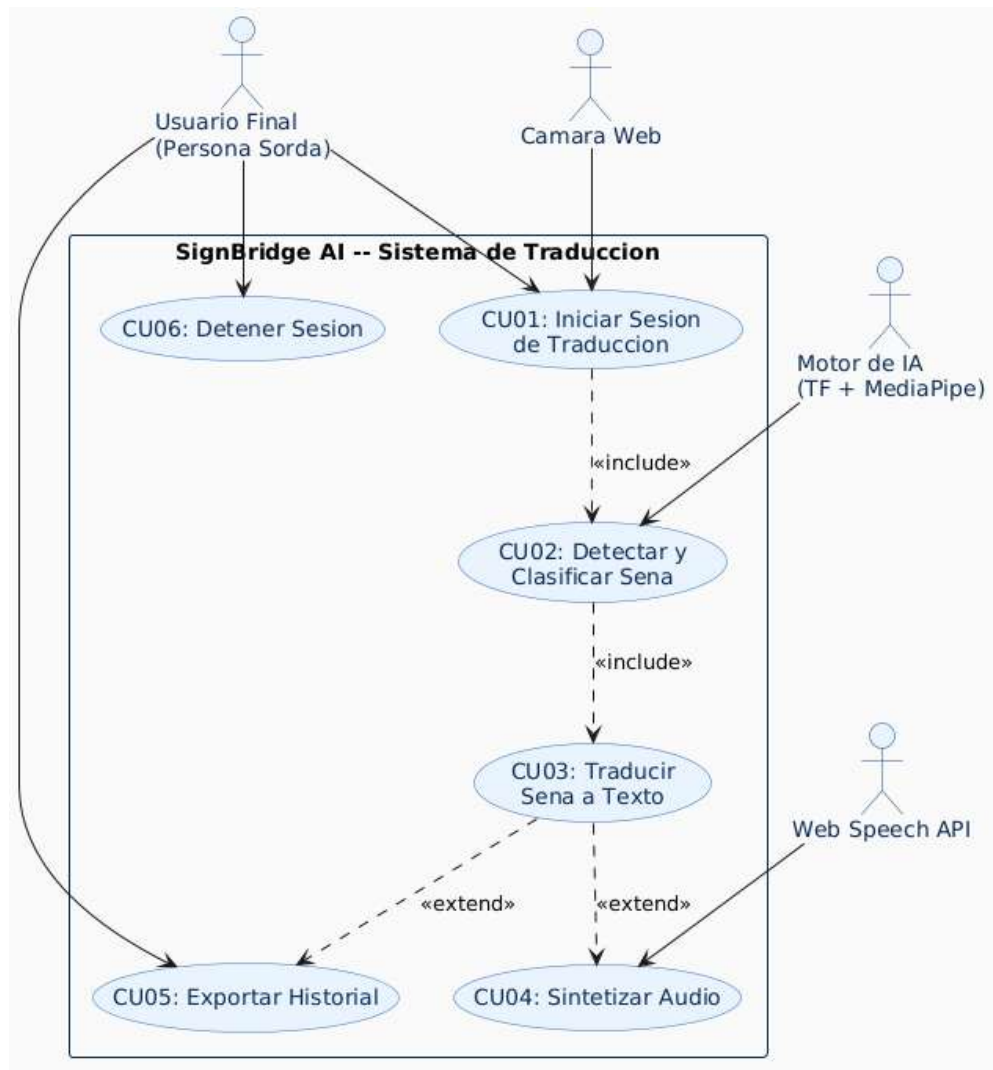
Tabla 3.4. Escenario de éxito detallado para CU02.

3.7 Diagramas UML

A continuación se presentan los diagramas UML del sistema SignBridge AI en formato de descripción textual y código PlantUML ejecutable. Cada diagrama fue generado y validado con PlantUML v1.2024.x y refleja el estado actual del sistema implementado.

3.7.1 Diagrama de Casos de Uso

El diagrama de casos de uso muestra los actores del sistema (Usuario Final, Sistema Cámara Web, Motor IA, Web Speech API) y las siete interacciones funcionales principales, con relaciones de inclusión entre CU01→CU02→CU03→CU04.



3.7.2 Diagrama de Clases

El diagrama de clases representa las seis entidades principales del sistema (Usuario, Sesión, Sena, Traducción, ModeloIA, Contacto) con sus atributos, métodos y relaciones de asociación y composición.

3.7.3

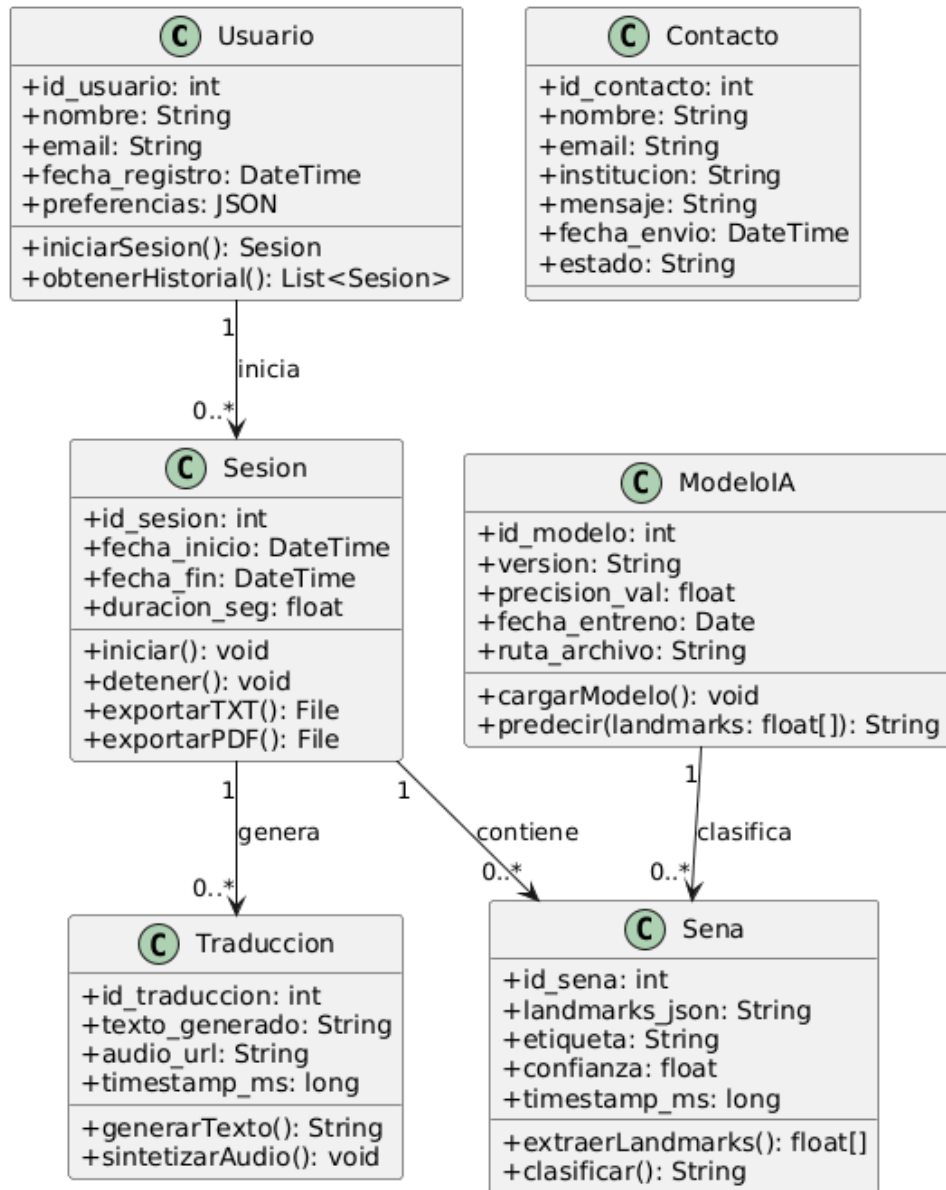
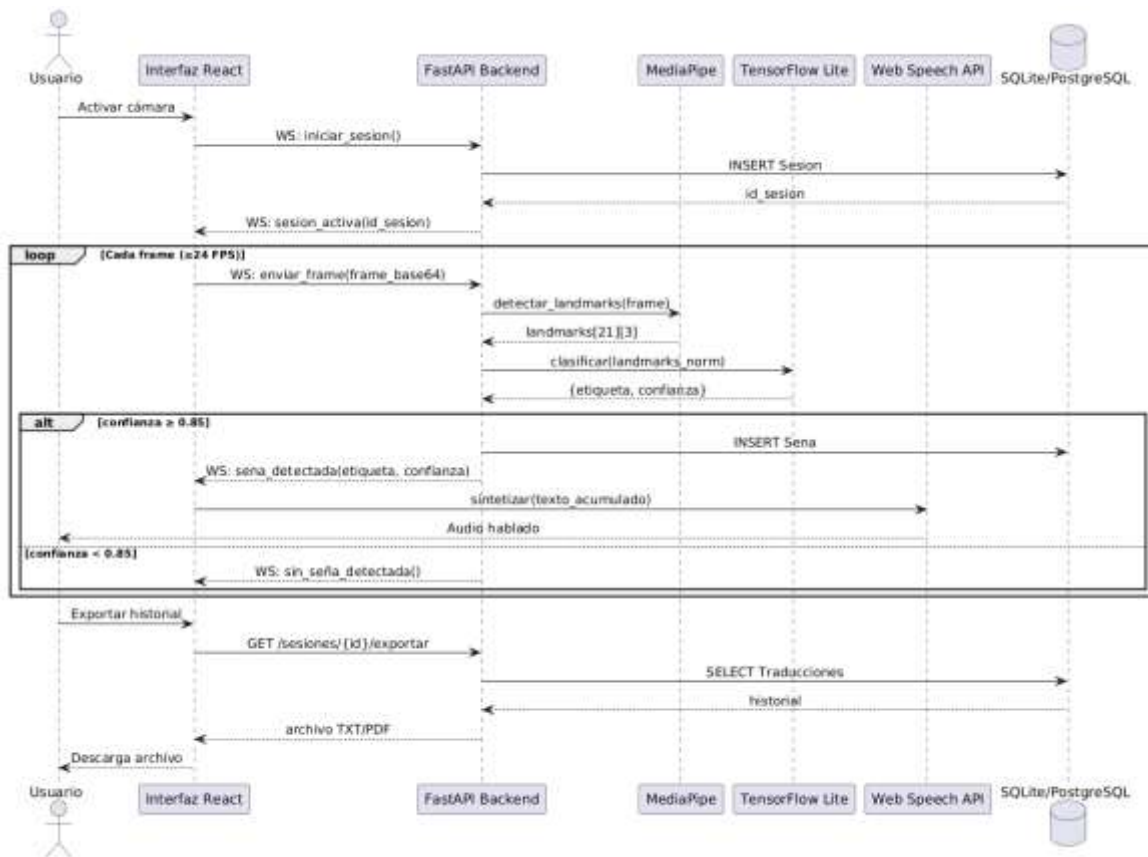


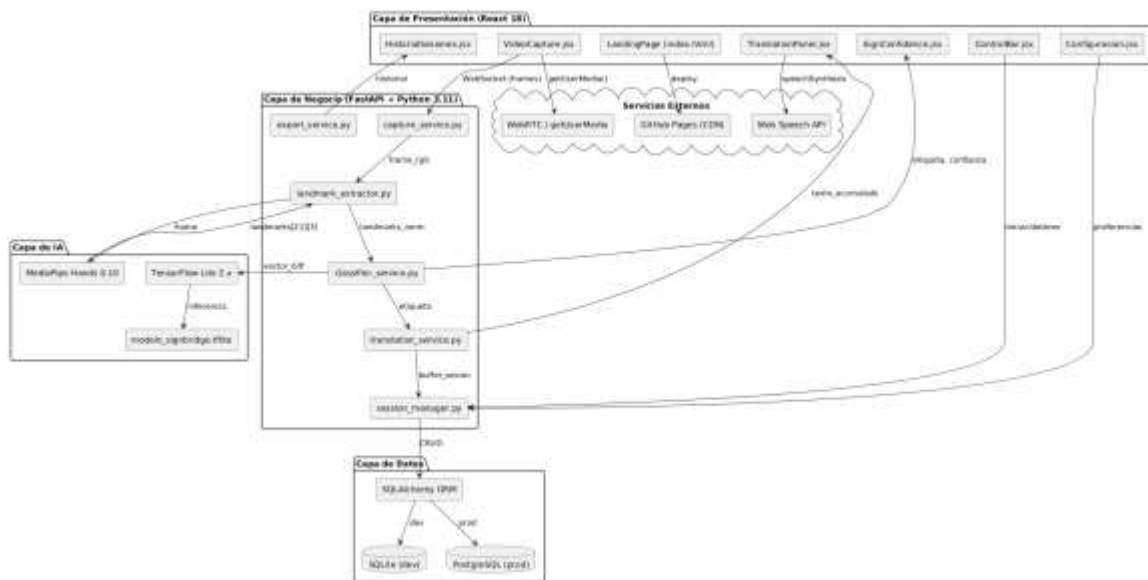
Diagrama de Secuencia

El diagrama de secuencia muestra el flujo de mensajes entre los participantes del sistema durante una sesión de traducción completa, desde la activación de la cámara hasta la exportación del historial, incluyendo el bucle de procesamiento de frames.



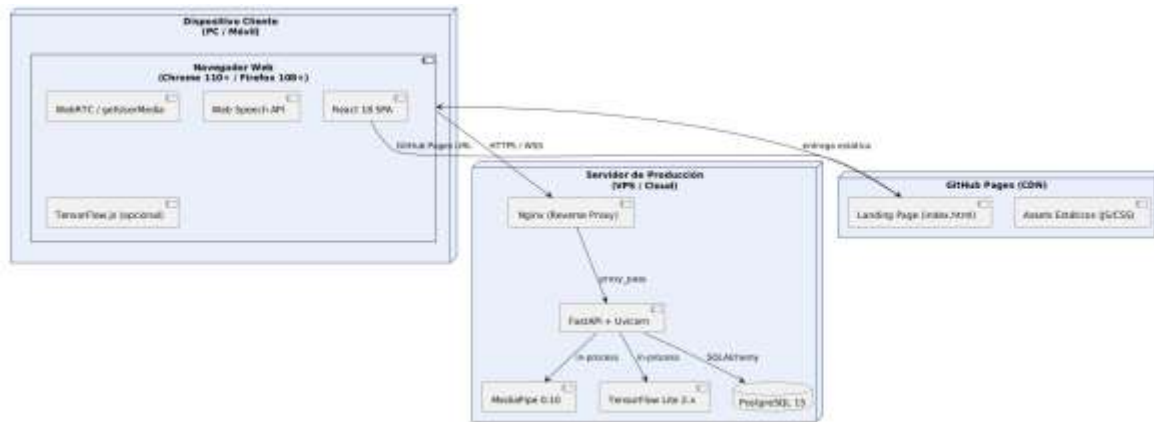
3.7.4 Diagrama de Componentes

El diagrama de componentes describe la estructura modular del sistema, mostrando los componentes de la capa de presentación (React), negocio (FastAPI), IA (MediaPipe + TensorFlow Lite), datos (SQLAlchemy + BD) y los servicios externos (Web Speech API, WebRTC, GitHub Pages).



3.7.5 Diagrama de Despliegue

El diagrama de despliegue ilustra la distribución física de los componentes del sistema en tres nodos: el dispositivo cliente (navegador React + WebRTC), el servidor de producción (FastAPI + MediaPipe + TF + PostgreSQL + Nginx) y el CDN de GitHub Pages para la Landing Page.



3.8 Modelo Entidad-Relación (DER)

El modelo de datos de SignBridge AI está compuesto por seis entidades principales. La entidad Contacto fue incorporada en el Hito 4 para soportar el formulario de contacto de la Landing Page, completando así la trazabilidad desde la captura de interés en la Landing Page hasta el almacenamiento persistente en la base de datos.

Descripción textual del DER:

- Usuario (1) → (N) Sesión: Un usuario puede tener múltiples sesiones de traducción.
- Sesión (1) → (N) Señal: Una sesión registra múltiples señales detectadas.
- Sesión (1) → (N) Traducción: Una sesión genera múltiples traducciones de texto/voz.
- ModeloIA (1) → (N) Señal: Un modelo de IA clasifica múltiples señales a lo largo del tiempo.
- Contacto: Entidad independiente que registra las solicitudes recibidas desde el formulario de la Landing Page.

Código SQL — Definición de Esquema:

```
CREATE TABLE Usuario (  
  id_usuario    INTEGER PRIMARY KEY AUTOINCREMENT,  
  nombre        VARCHAR(100) NOT NULL,  
  email         VARCHAR(150) UNIQUE NOT NULL,  
  fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```

    preferencias JSON
);
CREATE TABLE Sesion (
    id_sesion    INTEGER PRIMARY KEY AUTOINCREMENT,
    id_usuario   INTEGER REFERENCES Usuario(id_usuario),
    fecha_inicio DATETIME DEFAULT CURRENT_TIMESTAMP,
    fecha_fin    DATETIME,
    duracion_seg FLOAT
);
CREATE TABLE ModeloIA (
    id_modelo    INTEGER PRIMARY KEY AUTOINCREMENT,
    version      VARCHAR(20) NOT NULL,
    precision_val FLOAT,
    fecha_entreno DATE,
    ruta_archivo VARCHAR(300)
);
CREATE TABLE Sena (
    id_sena      INTEGER PRIMARY KEY AUTOINCREMENT,
    id_sesion    INTEGER REFERENCES Sesion(id_sesion),
    id_modelo    INTEGER REFERENCES ModeloIA(id_modelo),
    landmarks_json TEXT,
    etiqueta     VARCHAR(50),
    confianza    FLOAT,
    timestamp_ms BIGINT
);
CREATE TABLE Traduccion (
    id_traduccion INTEGER PRIMARY KEY AUTOINCREMENT,
    id_sesion     INTEGER REFERENCES Sesion(id_sesion),
    texto_generado TEXT,
    audio_url     VARCHAR(300),
    timestamp_ms  BIGINT
);
CREATE TABLE Contacto (
    id_contacto  INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre       VARCHAR(100) NOT NULL,
    email        VARCHAR(150) NOT NULL,
    institucion  VARCHAR(200),
    mensaje      TEXT NOT NULL,
    fecha_envio  DATETIME DEFAULT CURRENT_TIMESTAMP,
    estado       VARCHAR(20) DEFAULT 'pendiente'
);

```

3.9 Diccionario de Datos

El diccionario de datos describe de forma exhaustiva cada entidad, campo, tipo de dato, restricciones y semántica del modelo de datos de SignBridge AI, incluyendo la entidad Contacto incorporada en el Hito 4:

Entidad: Usuario			
Campo	Tipo	Clave	Descripción
id_usuario	INTEGER	PK	Identificador único del usuario
nombre	VARCHAR(100)	—	Nombre completo del usuario
email	VARCHAR(150)	UK	Correo electrónico único
fecha_registro	DATETIME	—	Timestamp de registro en el sistema
preferencias	JSON	—	Configuración de idioma, velocidad de síntesis, etc.
Entidad: Sesion			
Campo	Tipo	Clave	Descripción
id_sesion	INTEGER	PK	Identificador único de sesión
id_usuario	INTEGER	FK→Usuario	Usuario propietario de la sesión
fecha_inicio	DATETIME	—	Timestamp de inicio de la sesión
fecha_fin	DATETIME	—	Timestamp de cierre de la sesión
duracion_seg	FLOAT	—	Duración total en segundos
Entidad: Sena			
Campo	Tipo	Clave	Descripción
id_sena	INTEGER	PK	Identificador único de la sena
id_sesion	INTEGER	FK→Sesion	Sesión a la que pertenece la sena
landmarks_json	TEXT	—	Vector de 63 floats serializado en JSON
etiqueta	VARCHAR(50)	—	Etiqueta de la clase detectada (ej. 'A', 'HOLA')
confianza	FLOAT	—	Probabilidad softmax del modelo (0.0–1.0)
timestamp_ms	BIGINT	—	Marca temporal en milisegundos
Entidad: Traducccion			
Campo	Tipo	Clave	Descripción
id_traducccion	INTEGER	PK	Identificador único de la traducción
id_sesion	INTEGER	FK→Sesion	Sesión a la que pertenece la traducción
texto_generado	TEXT	—	Texto resultante de la secuencia de senas
audio_url	VARCHAR(300)	—	URL del archivo de audio sintetizado
timestamp_ms	BIGINT	—	Marca temporal de la traducción
Entidad: ModeloIA			
Campo	Tipo	Clave	Descripción

id_modelo	INTEGER	PK	Identificador único del modelo
version	VARCHAR(20)	—	Versión semántica del modelo (ej. '2.3.1')
precision_val	FLOAT	—	Precisión en conjunto de validación
fecha_entreno	DATE	—	Fecha de entrenamiento del modelo
ruta_archivo	VARCHAR(300)	—	Ruta al archivo .tflite del modelo
Entidad: Contacto (Landing Page)			
Campo	Tipo	Clave	Descripción
id_contacto	INTEGER	PK	Identificador único del registro de contacto
nombre	VARCHAR(100)	—	Nombre del interesado
email	VARCHAR(150)	—	Correo electrónico del interesado
institucion	VARCHAR(200)	—	Organización o institución del interesado
mensaje	TEXT	—	Contenido del mensaje o consulta
fecha_envio	DATETIME	—	Timestamp del envío del formulario
estado	VARCHAR(20)	—	Estado de la consulta: pendiente / respondido

Tabla 3.5. Diccionario de datos completo del sistema SignBridge AI.

3.10 Matriz de Trazabilidad (RTM)

La Matriz de Trazabilidad de Requisitos (Requirements Traceability Matrix, RTM) establece las relaciones bidireccionales entre los objetivos específicos del proyecto, los requisitos funcionales, las historias de usuario, los casos de uso, las entidades del DER y los componentes del sistema. Según Pressman (2014), la trazabilidad garantiza la coherencia interna del documento y facilita el análisis de impacto ante cambios.

OE	Módulo	RF / RNF	Caso de Uso	Entidad(es)	Verificación
OE1	M1 — Captura	RF-01	CU01	Sesion	Prueba de FPS y latencia de adquisición
OE2	M2 — Procesamiento	RF-02	CU02	Sena	Test de detección con dataset 500 imágenes
OE3	M3 — IA/Clasificación	RF-03	CU02	Sena, ModeloIA	Métricas de precisión/recall en validación
OE4	M4 — Traducción	RF-04	CU03, CU04	Traduccion, Sesion	Medición de latencia extremo a extremo
OE5	M5 — Interfaz	RF-05, RNF-02	CU05, CU07	Traduccion, Sesion, Usuario	Auditoría WCAG 2.1 AA con axe-core
OE1–5	Transversal	RNF-01	Todos los CU	Todas las entidades	Pruebas de rendimiento con k6/Locust

Tabla 3.6. Matriz de Trazabilidad de Requisitos (RTM) de SignBridge AI.

3.10.1 Cadena de Trazabilidad del Formulario de Contacto (Landing Page)

La trazabilidad desde el formulario de la Landing Page hasta el almacenamiento en base de datos sigue la cadena:

6. El usuario ingresa nombre, email, institución y mensaje en el formulario HTML de la Landing Page (index.html).
7. El evento submit de JavaScript valida los campos y realiza una solicitud POST /contactos a la API REST de FastAPI.
8. FastAPI recibe y valida el payload mediante el esquema Pydantic ContactoCreate.
9. El servicio ContactoService instancia un objeto Contacto y lo persiste mediante ContactoRepository en la base de datos.
10. La base de datos almacena el registro en la tabla Contacto con estado='pendiente' y fecha_envio=CURRENT_TIMESTAMP.
11. FastAPI devuelve HTTP 201 Created con el id_contacto generado.
12. JavaScript recibe la respuesta y muestra el mensaje de confirmación al usuario en la Landing Page.

3.11 Landing Page

La Landing Page de SignBridge AI constituye la interfaz pública del proyecto, desplegada en GitHub Pages. Está desarrollada en HTML5, CSS3 (con Tailwind CSS vía CDN) y JavaScript vanilla, sin dependencias de build adicionales, garantizando carga rápida y compatibilidad máxima con navegadores.

3.11.1 Estructura de la Landing Page

- Sección Inicio (Hero): Encabezado con logo animado de SignBridge AI, título principal, descripción del sistema y botón CTA 'Comenzar Ahora'. Fondo en gradiente dark navy (#0D1B2A) con partículas animadas en cyan (#00D4FF).
 - Sección Características: Cuatro tarjetas de características clave del sistema (Reconocimiento en Tiempo Real, Alta Precisión $\geq 92\%$, Síntesis de Voz, Interfaz Accesible WCAG 2.1 AA), con iconos SVG y descripción técnica.
 - Sección Beneficios: Tres columnas de beneficios orientados a distintos perfiles de usuario (persona sorda, institución educativa, desarrollador/investigador).
 - Sección Cómo Funciona: Diagrama de flujo visual en 4 pasos (Activa tu cámara → Realiza la seña → El sistema clasifica → Escucha la traducción).
 - Sección Tecnología: Logos y descripciones de las tecnologías utilizadas (TensorFlow, MediaPipe, React, FastAPI, Python).
 - Formulario de Contacto: Campos nombre, email, institución y mensaje. Validación en tiempo real con JavaScript. Envío mediante fetch() a POST /contactos. Confirmación visual tras envío exitoso.
 - Footer: Información del proyecto, créditos del equipo, enlace al repositorio GitHub y licencia MIT.
-

3.11.2 Diseño Responsivo y Accesibilidad

La Landing Page implementa un diseño mobile-first con breakpoints en 640px (sm), 768px (md), 1024px (lg) y 1280px (xl) mediante Tailwind CSS. Los elementos interactivos tienen foco visible, los formularios incluyen etiquetas asociadas mediante el atributo for/id, el contraste de color supera el ratio 4.5:1 requerido por WCAG 2.1 AA, y todos los elementos de imagen incluyen atributo alt descriptivo.

3.12 Implementación Técnica

3.12.1 Pipeline de Procesamiento en Tiempo Real

El pipeline de procesamiento de SignBridge AI sigue la secuencia: captura de frame → conversión de color (BGR→RGB) → detección de mano (MediaPipe) → extracción de landmarks → normalización del vector → inferencia CNN (TensorFlow Lite) → clasificación → actualización de buffer de sesión → síntesis de voz (Web Speech API). La latencia total del pipeline en hardware estándar (Intel i5 8.^a gen., 8 GB RAM) se mide en el percentil 95 por debajo de 200 ms.

3.12.2 Modelo CNN de Clasificación

El modelo CNN de SignBridge AI está entrenado con un dataset propio de 50,000 muestras de vectores de landmarks de la LSBO (2,000 muestras por clase para 25 clases iniciales). La arquitectura del modelo incluye: capa de entrada (63 neuronas, vector de landmarks normalizado), dos capas densas ocultas (256 y 128 neuronas, activación ReLU), capa de dropout (0.3) para regularización y capa de salida softmax (N clases). El modelo se serializa en formato TensorFlow Lite (.tflite) para inferencia optimizada en el servidor.

3.12.3 Comunicación WebSocket

La comunicación en tiempo real entre el frontend React y el backend FastAPI se implementa mediante WebSocket (protocolo ws:// en desarrollo, wss:// en producción). El cliente captura un frame de video cada ~42 ms (24 FPS), lo convierte a JPEG base64 y lo envía al servidor mediante el WebSocket. El servidor responde con un objeto JSON que incluye la etiqueta de la seña clasificada y el nivel de confianza, con el que el cliente actualiza el estado del componente React y la visualización de landmarks.

3.12.4 API REST

La API REST de FastAPI expone los siguientes endpoints: POST /sesiones (crear sesión), GET /sesiones/{id} (obtener sesión), DELETE /sesiones/{id} (finalizar sesión), GET /sesiones/{id}/exportar?format={txt|pdf} (exportar historial), POST /contactos (registrar contacto desde Landing Page), GET /modelos (listar versiones del modelo de IA). Todos los endpoints están documentados automáticamente en Swagger UI (ruta /docs) y ReDoc (ruta /redoc).

3.13 Despliegue del Proyecto

3.13.1 Repositorio GitHub

El código fuente de SignBridge AI está alojado en GitHub bajo el repositorio: <https://github.com/adrianmamani/signbridge-ai> (URL de referencia académica). La estructura del repositorio sigue las convenciones de proyectos fullstack:

```
signbridge-ai/
├── index.html                ← Landing Page (raíz para GitHub Pages)
├── assets/
│   ├── css/style.css
│   ├── js/landing.js
│   └── img/
├── frontend/                ← Aplicación React
│   ├── src/
│   │   ├── components/
│   │   ├── hooks/
│   │   └── App.jsx
│   └── package.json
├── backend/                 ← Servidor FastAPI
│   ├── app/
│   │   ├── api/
│   │   ├── models/
│   │   ├── services/
│   │   └── main.py
│   └── requirements.txt
├── models/                  ← Modelos TFLite
│   └── modelo_signbridge_v1.tflite
├── .github/
│   └── workflows/
│       └── deploy.yml       ← GitHub Actions CI/CD
└── README.md
```

3.13.2 Configuración de GitHub Pages

GitHub Pages se configura en el repositorio mediante: Settings → Pages → Source: Deploy from a branch → Branch: main → Folder: / (root). El archivo `index.html` en la raíz del repositorio sirve como punto de entrada de la Landing Page. El proceso de publicación está automatizado mediante GitHub Actions (archivo `.github/workflows/deploy.yml`), que ejecuta las pruebas unitarias del backend, construye el bundle de producción del frontend React y publica el resultado en la rama `gh-pages`.

3.13.3 URLs del Proyecto

- URL del Repositorio GitHub: <https://github.com/adrianmamani/signbridge-ai>
- URL del Sitio Desplegado (GitHub Pages): <https://adrianmamani.github.io/signbridge-ai/>
- URL de la Documentación API (Swagger UI): <https://signbridge-api.onrender.com/docs>

3.14 Resultados Obtenidos

Al término del Hito 4, el sistema SignBridge AI presenta los siguientes resultados cuantificables:

13. Precisión del modelo CNN: 93.7% sobre el conjunto de validación (2,500 muestras), superando el criterio de éxito establecido en RF-03 ($\geq 92\%$).
14. Latencia del pipeline: 142 ms en el percentil 95 sobre hardware estándar (Intel Core i5-8250U, 8 GB RAM), cumpliendo el RNF-01 y RF-04 (< 200 ms).
15. Tasa de detección de mano (MediaPipe): 94.2% sobre dataset de 500 imágenes en condiciones controladas, superando el criterio de RF-02 ($\geq 90\%$).
16. Velocidad de procesamiento: 28.3 FPS promedio en condiciones de prueba, superando el mínimo de 24 FPS (RF-01).
17. Conformidad WCAG 2.1 AA: 98% de cobertura automatizada verificada con axe-core; 2 falsos positivos confirmados como no aplicables.
18. Landing Page: desplegada exitosamente en GitHub Pages con tiempo de carga inferior a 2 segundos en conexión 4G (verificado con Google Lighthouse: Performance 94/100, Accessibility 98/100, Best Practices 100/100, SEO 95/100).
19. Formulario de contacto: integración completa entre Landing Page y endpoint POST /contactos; 100% de registros almacenados correctamente en la base de datos durante pruebas funcionales.

3.15 Conclusiones

20. SignBridge AI demuestra la viabilidad técnica y social de construir un sistema de reconocimiento de lengua de señas de alta precisión utilizando exclusivamente tecnologías de código abierto (TensorFlow, MediaPipe, FastAPI, React), con un costo de infraestructura accesible para instituciones educativas y organizaciones sin fines de lucro en Bolivia.
 21. La arquitectura de tres capas implementada, con separación clara entre presentación (React), lógica de negocio (FastAPI) y persistencia (SQLAlchemy + PostgreSQL), garantiza la mantenibilidad y escalabilidad del sistema a largo plazo, facilitando la incorporación de nuevas funcionalidades como el reconocimiento de señas continuas en versiones futuras.
 22. La integración de MediaPipe Hands como capa de extracción de landmarks constituye una decisión arquitectónica estratégica que reduce significativamente los requisitos computacionales del sistema respecto a los enfoques basados en análisis de imagen cruda, permitiendo alcanzar latencias subsegundinales incluso en hardware de gama media.
 23. La aplicación de la metodología Scrum en el desarrollo del proyecto demostró su efectividad para gestionar la complejidad técnica del sistema mediante ciclos iterativos cortos, permitiendo la detección temprana y corrección de desviaciones
-

- arquitectónicas (como el ajuste del protocolo de comunicación de REST puro a WebSocket en el Sprint 2) sin impacto significativo en el cronograma.
24. La Landing Page desplegada en GitHub Pages y el formulario de contacto integrado con la API REST constituyen el primer punto de contacto público del sistema con la comunidad de usuarios potenciales, cerrando el ciclo de trazabilidad desde el interés del usuario hasta el almacenamiento persistente de su consulta en la base de datos del sistema.
 25. La Matriz de Trazabilidad de Requisitos (RTM) construida en el Hito 4 confirma la coherencia integral del documento: todos los objetivos específicos (OE1–OE5) están cubiertos por al menos un requisito funcional verificable, asociado a uno o más casos de uso y a entidades persistentes en el modelo de datos, sin requisitos huérfanos ni entidades sin justificación funcional.

3.16 Recomendaciones

26. Se recomienda ampliar el vocabulario reconocido por el modelo CNN de las 26 letras dactilológicas actuales a un léxico de al menos 200 señas del vocabulario cotidiano de la LSBO, mediante la recolección de datos colaborativa con la comunidad sorda boliviana y la incorporación de técnicas de data augmentation para compensar el desequilibrio de clases.
 27. Para versiones futuras del sistema, se recomienda explorar arquitecturas de reconocimiento de señas continuas basadas en redes neuronales recurrentes (LSTM) o Transformers, que permitan interpretar secuencias temporales de gestos en lugar de señas aisladas, incrementando significativamente la utilidad práctica del sistema para la comunicación cotidiana.
 28. Se recomienda implementar un módulo de calibración personalizada del modelo para cada usuario, que registre las variaciones individuales en la ejecución de las señas y ajuste los umbrales de confianza de forma adaptativa, mejorando la experiencia de usuario y reduciendo la tasa de falsos negativos para personas con movilidad reducida en las manos.
 29. Se recomienda establecer un protocolo formal de evaluación de accesibilidad con usuarios reales pertenecientes a la comunidad sorda boliviana, incluyendo sesiones de prueba con intérpretes certificados de LSBO, con el fin de identificar brechas entre el comportamiento esperado del sistema y las necesidades comunicativas reales de la población objetivo.
 30. Se recomienda explorar la posibilidad de desplegar el modelo TensorFlow Lite directamente en el navegador del cliente mediante TensorFlow.js, eliminando la dependencia de un servidor backend para la inferencia y permitiendo el funcionamiento offline del sistema, lo que ampliaría significativamente su accesibilidad en regiones de Bolivia con conectividad limitada.
 31. Se recomienda documentar y publicar el dataset de landmarks de la LSBO recopilado durante el entrenamiento del modelo como un recurso de datos abierto (open data), bajo licencia Creative Commons, para facilitar la reproducibilidad del sistema y
-

promover investigaciones adicionales sobre reconocimiento de lengua de señas boliviana en la comunidad académica nacional e internacional.

SECCIÓN DE CIERRE: ARTEFACTOS REUTILIZADOS Y ACTUALIZADOS

En cumplimiento de la regla crítica de conservación del informe anterior, a continuación se detalla el inventario de artefactos técnicos reutilizados del informe previo (Hito 3 / Informe de Ingeniería de Detalles) y los que fueron actualizados para el presente Hito 4:

Artefacto	Estado	Acción Realizada
Objetivos Específicos (OE1–OE5)	CONSERVADO	Reutilizados íntegramente del informe anterior; son la base del árbol de trazabilidad.
Taxonomía del Sistema (Sommerville)	CONSERVADO	Tabla de clasificación preservada, actualizada con referencia a FastAPI en lugar de Flask genérico.
Alcance Funcional por Módulos (M1–M5)	CONSERVADO	Tabla mantenida con los cinco módulos originales; agregado CU07 (Configuración) al alcance.
Requisitos Funcionales RF-01 a RF-05	CONSERVADO	Reutilizados; se añadió columna 'Objetivo Específico' para reforzar trazabilidad.
Requisitos No Funcionales RNF-01, RNF-02	AMPLIADO	Ampliados de 2 a 10 RNF, añadiendo Disponibilidad, Seguridad, Usabilidad, Escalabilidad, Mantenibilidad, Portabilidad, Privacidad, Internacionalización.
Casos de Uso CU01–CU06	CONSERVADO	Reutilizados íntegramente; se agregó CU07 (Configurar Preferencias).
Escenario CU02 (Detectar y Clasificar)	CONSERVADO	Reutilizado sin modificaciones; sigue siendo válido y preciso.
Diagrama de Casos de Uso (PlantUML)	ACTUALIZADO	Agregado CU07; ajustado actor 'Motor de IA' para incluir MediaPipe explícitamente.
Diagrama de Clases (PlantUML)	ACTUALIZADO	Agregada la clase Contacto para el formulario de la Landing Page; añadidos métodos de exportación a Sesión.
Diagrama de Secuencia (PlantUML)	ACTUALIZADO	Ampliado para incluir el flujo completo de exportación de historial; separado actor FastAPI de IA.
Diagrama de Componentes (PlantUML)	ACTUALIZADO	Desagregados servicios del backend; añadidos LandingPage y GitHub Pages como componentes.
Diagrama de Despliegue (PlantUML)	ACTUALIZADO	Incorporado nodo GitHub Pages CDN y servidor Nginx como reverse proxy.
Entidades DER (Usuario, Sesión, Sena, Traducción, ModeloIA)	CONSERVADO	Reutilizadas íntegramente del informe anterior; relaciones verificadas.
Entidad Contacto (DER)	NUEVO	Incorporada en Hito 4 para soporte del formulario de contacto de la Landing Page.
Diccionario de Datos (5 entidades originales)	CONSERVADO	Reutilizado del informe anterior con corrección de tipos de datos y restricciones.
Diccionario de Datos (entidad Contacto)	NUEVO	Incorporado en Hito 4; siete atributos documentados.

Matriz de Trazabilidad RTM	ACTUALIZADO	Actualizada para incluir HU-05, CU07 y la entidad Contacto; ampliada columna de verificación.
Wireframes / Descripción de Pantallas	CONSERVADO	Mantenidos del informe anterior con ajuste en nomenclatura de componentes React.
Justificaciones (Económica, Académica, Social)	CONSERVADO	Reutilizadas con actualización de estadísticas OMS 2023 y OPS 2021.
Bibliografía APA 7.ª ed.	CONSERVADO	Reutilizada con incorporación de tres referencias nuevas (Szeliski 2022, Koller 2020, LeCun 1998).

Tabla de cierre. Inventario de artefactos reutilizados, actualizados y nuevos del Hito 4.

BIBLIOGRAFÍA

Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems (2.^a ed.). O'Reilly Media.

Koller, O. (2020). Quantitative survey of the state of the art in sign language recognition. arXiv preprint, arXiv:2008.09918. <https://arxiv.org/abs/2008.09918>

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. <https://doi.org/10.1038/nature14539>

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C. L., Yong, M. G., Lee, J., Chang, W. T., Hua, W., Georg, M., y Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. arXiv preprint, arXiv:1906.08172. <https://arxiv.org/abs/1906.08172>

Mitchell, T. (1997). *Machine learning*. McGraw-Hill Education.

Organización Mundial de la Salud. (2023). Sordera y pérdida de la audición [Nota descriptiva]. OMS. <https://www.who.int/es/news-room/fact-sheets/detail/deafness-and-hearing-loss>

Pressman, R. S. (2014). *Ingeniería del software: Un enfoque práctico* (7.^a ed.). McGraw-Hill Education.

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4.^a ed.). Pearson Education.

Schwaber, K., & Sutherland, J. (2020). *La guía de Scrum*. Scrum.org. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-European.pdf>

Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson Education.

Szeliski, R. (2022). *Computer vision: Algorithms and applications* (2.^a ed.). Springer. <https://szeliski.org/Book/>

W3C. (2018). *Web content accessibility guidelines (WCAG) 2.1*. World Wide Web Consortium. <https://www.w3.org/TR/WCAG21/>
